Relatório de Desempenho

Comparação: InsertSort vs QuickSort

Testes com Vetores Ordenados e Invertidos

Aluno: João Vitor Nunes Lopes

Github: https://github.com/jvitorLS/trabs-complexidade-algoritmo

Professor: Elgio Schlemer

Disciplina: Complexidade e Avaliação de Desempenho

Linguagem: C

Tabela de Resultados

Tamanho	Caso	InsertSort (us)	QuickSort (us)
5000	Ordenado	531	473464
5000	Invertido	161286	189345
10000	Ordenado	258	1497609
10000	Invertido	689637	769874
15000	Ordenado	426	3333979
15000	Invertido	1725547	2070973
20000	Ordenado	697	6509736
20000	Invertido	3288261	3346785
25000	Ordenado	942	9703278
25000	Invertido	4610708	5285392

Análise dos Resultados

Os testes mostram claramente o impacto da complexidade assintótica no tempo de execução.

1) InsertSort
- Quando os dados já estão ordenados, o InsertSort praticamente não realiza trocas, operando próximo do melhor caso $O(n)$.
- Quando os dados estão invertidos, o algoritmo precisa deslocar muitos elementos a cada inserção, resultando no pior caso $O(n^2)$.
- Isso explica o crescimento muito rápido do tempo no cenário invertido.

2) QuickSort
- O QuickSort apresentou crescimento mais controlado, compatível com o caso médio $O(n \log n)$.
- Porém, dependendo da escolha do pivô, ele pode ter desempenho pior em vetores já ordenados, aproximando-se de $O(n^2)$.

3) Comparação Geral
- Para dados já ordenados, o InsertSort foi o mais rápido.
- Para dados invertidos e tamanhos maiores, o QuickSort foi significativamente mais eficiente.

Conclusão: InsertSort é adequado apenas para vetores pequenos ou quase ordenados, enquanto QuickSort é mais indicado
para grandes volumes de dados e casos gerais, mas pode degradar se o pivô não for escolhido adequadamente.

Gráfico 1 - Ordenação com Dados Ordenados

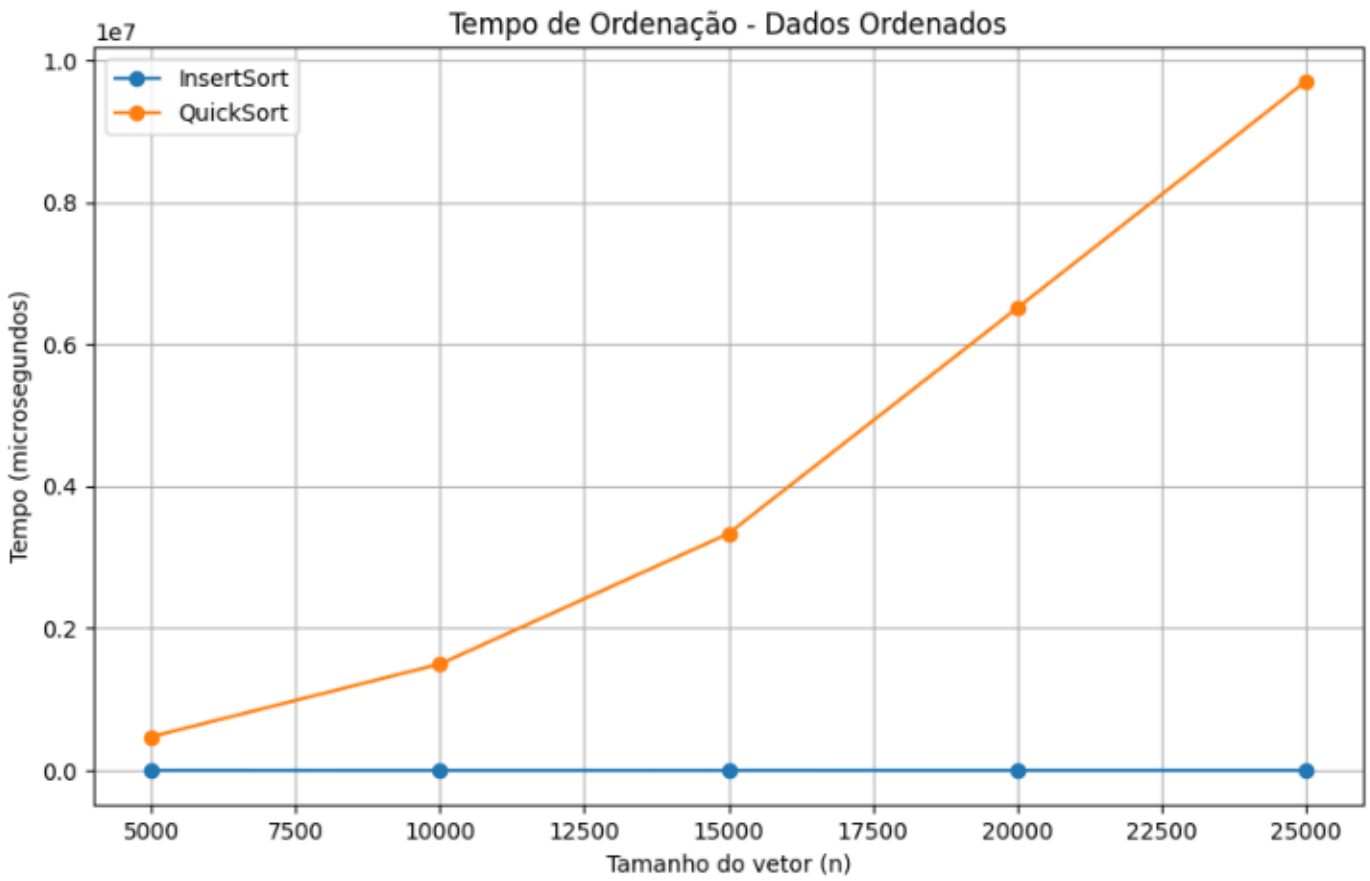

Gráfico 2 - Ordenação com Dados Invertidos

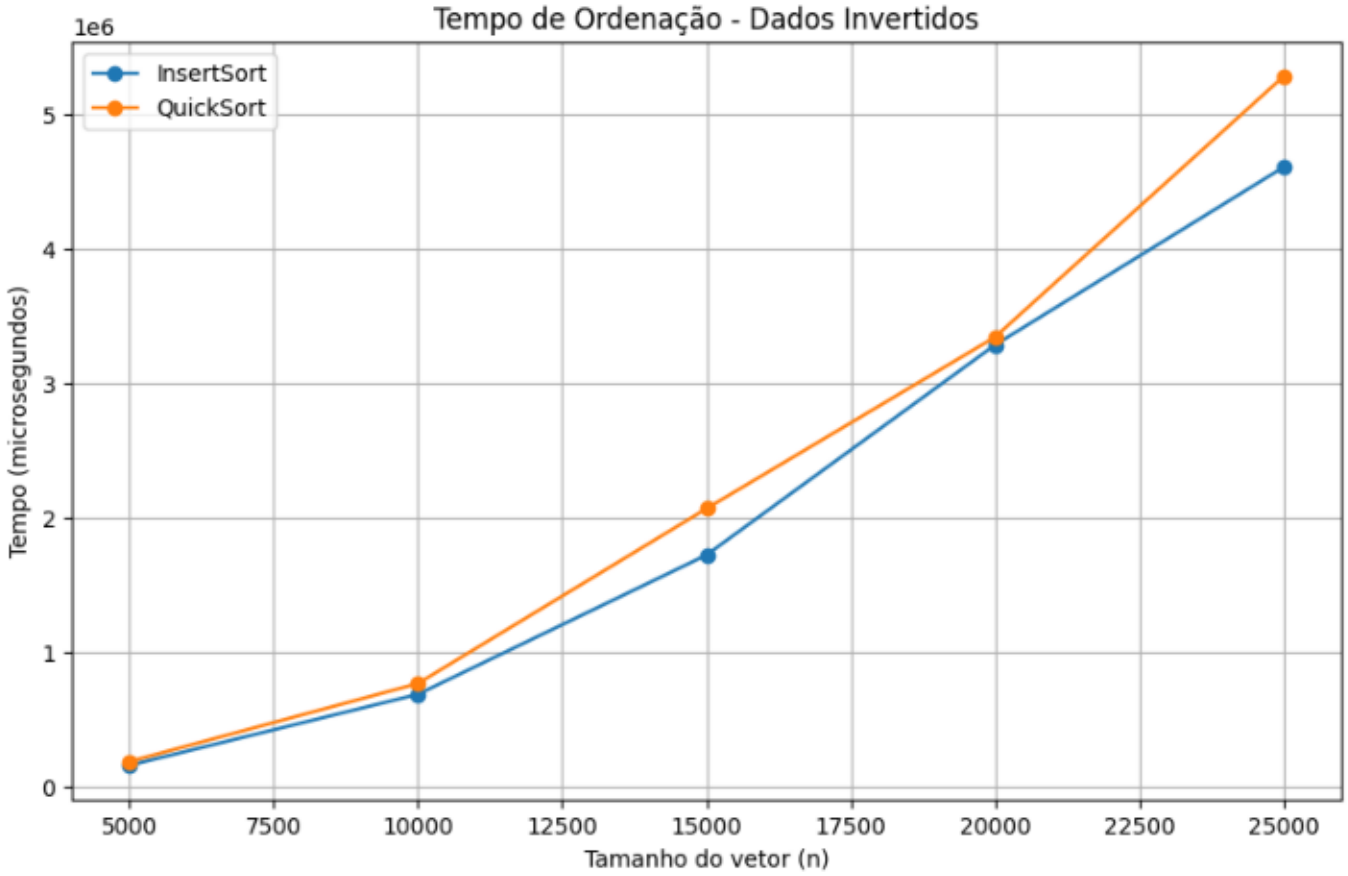

Gráfico 3 - InsertSort (Ordenado vs Invertido)

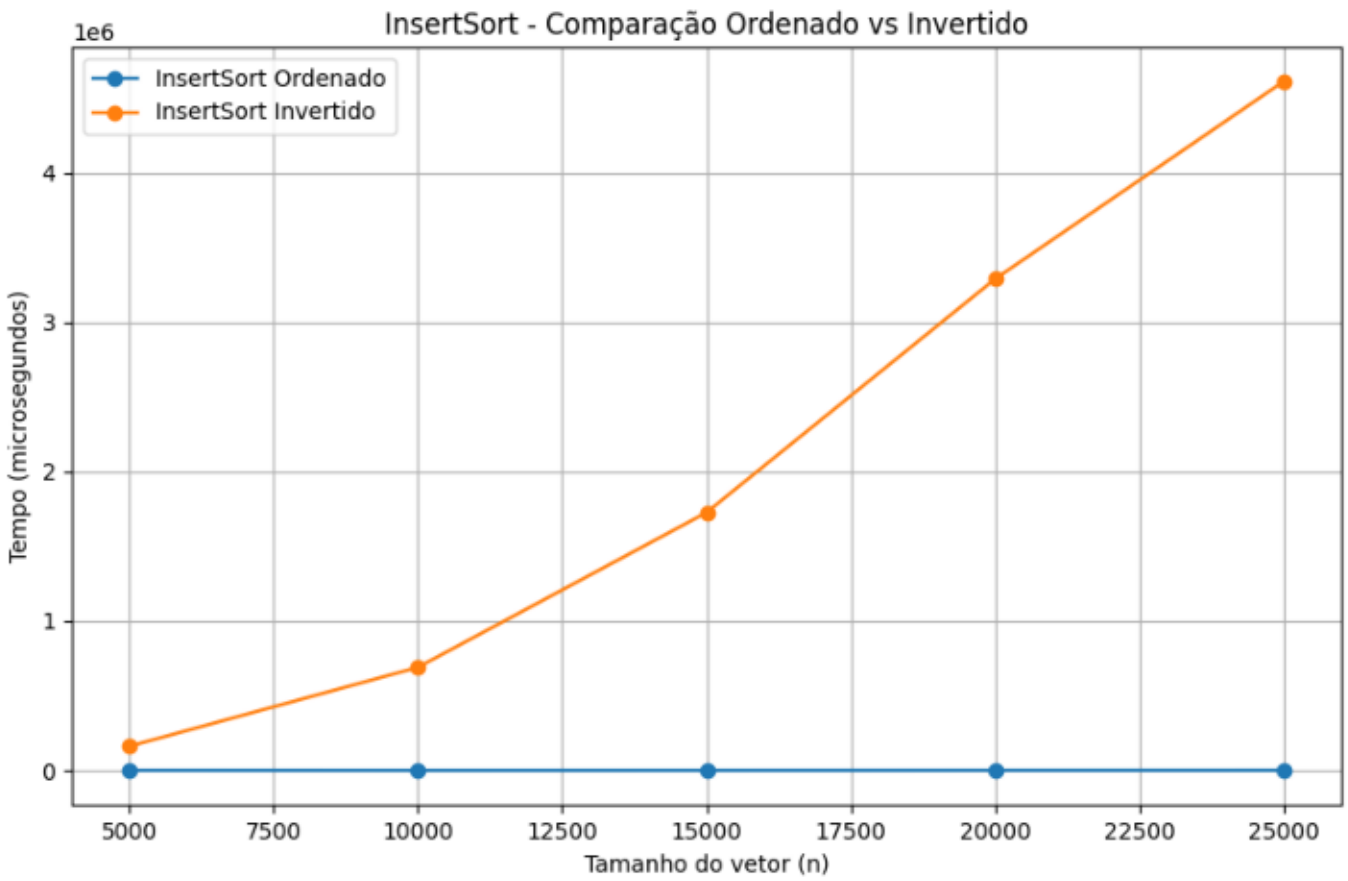
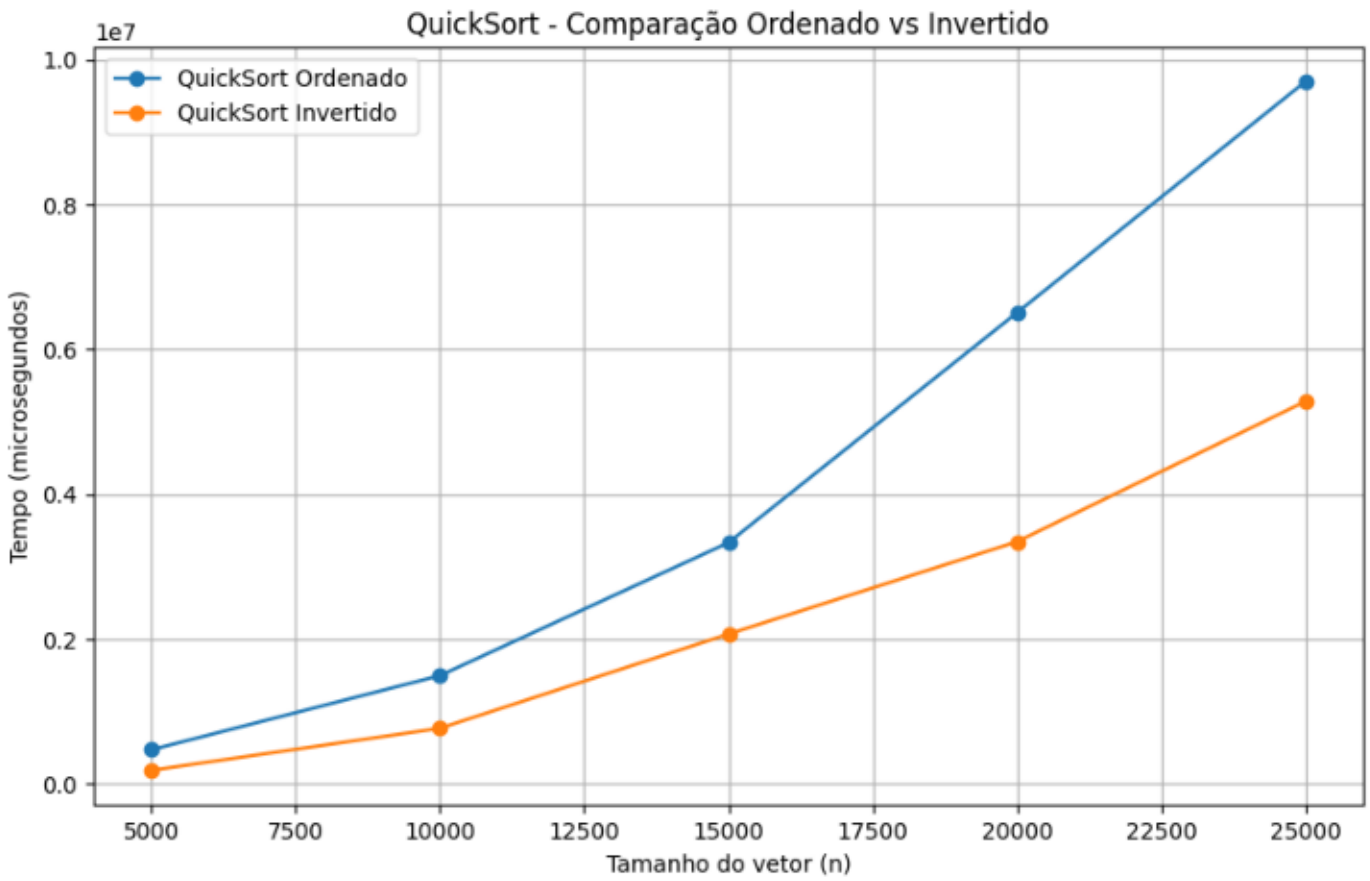
Gráfico 4 - QuickSort (Ordenado vs Invertido)